UNIVERSITY OF ZULULAND

FINAL%

Faculty of Science, Agriculture and Engineering

ASSIGNMENT COVER SHEET

MODULE TITLE	Assembly Language and Operating Systems
MODULE CODE	5EEE 481
LECTURER NAME	Dr.Oyebode
Topic	Project 8 – Instruction Encoder with Scheduling Simulation
DUE DATE	10 May 2026

NON - PLAGIARISM DECLARATION

I know that plagiarism means taking and using the ideas, writings, works or inventions of another as if they were one's own. I know that plagiarism not only includes verbatim copying, but also the extensive use of another person's ideas without proper acknowledgement (which includes the proper use of quotation marks). I know that plagiarism covers this sort of use of material found in textual sources and from the Internet. I acknowledge and understand that plagiarism is wrong. I understand that my research must be accurately referenced. I have followed the rules and conventions concerning referencing, citation and the use of quotations as set out in the Departmental Guide. This assignment is my own work, or my group's own unique group assignment. I acknowledge that copying someone else's assignment, or part of it, is wrong, and that submitting identical work to others constitutes a form of plagiarism. I have not allowed, nor will I in the future allow, anyone to copy my work with the intention of passing it off as their own work. By signing this cover sheet, I agree that I have read and understood the above. I acknowledge that should it be found to be higher than the acceptable similarity percentage, I may receive 0 (ZERO) for my assignment.

STUDENT NAME	STUDENT NO	
N Zungu	202375568	
LECTURER M REMARKS		

1. AI Tool Use Disclosure

For each AI tool that you have used, provide a brief, high-level summary:

1.	**Tool Name and Version:**	DeepSeek
	Purpose:	finding solution for errors
	Scope of Use	

2.	**Tool Name and Version:**	ChatGBT
	Purpose:	For finding the correct structure and code for report
	Scope of Use	

**Add as many tables as you require or delete tables as appropriate.*

2. AI Tool Use Detailed Disclosure by Category

For every instance where you used an AI tool, you must provide a detailed explanation.

A. AI as a Writing Tool (e.g., drafting, summarising, paraphrasing, editing, proofreading, image generation)

Requirement	*Explanation*
Specific use	Python code and Explanation of code
Input/Prompt	*prompt*
Validation process	
Required evidence	

Contents

List of figures

1. Introduction

This report is based on Project 8: Instruction Encoder with Scheduling Simulation. The project has two parts. Part A deals with ARM instruction encoding, while Part B deals with CPU scheduling using Python. The project guide states that Project 8 requires manual encoding of five ARM instructions and a scheduling simulation using FCFS and Round Robin [2].

This project supports GA1 because it requires identifying a computing problem, applying ARM and operating system principles, testing the solution, and giving conclusions based on results. The GA1 document explains that students must identify, analyse, and solve engineering problems using mathematics, engineering science, and computing principles [1]

2. Problem Statement

The problem is to understand how ARM instructions are represented at machine level and how an operating system schedules instruction stream. In real systems, the CPU executes encoded machine instructions, but the operating system decides which program gets CPU time.

The objectives are:

1. Write and test a small ARM program in CPUlator.
2. Encode five ARM instructions into 32-bit fields.
3. Explain condition codes, opcodes, registers, and operands.
4. Simulate FCFS and Round Robin scheduling in Python.
5. Compare waiting time and turnaround time.

3. Background Theory

Assembly language is a low-level language that represents machine code using readable instructions. It helps connect high-level programming to hardware operations [5]. The ARM notes explain that ARM assembly is useful for understanding computer architecture, instruction execution, memory management, and performance optimisation [5]. CPUlator was used because it allows ARM programs to be written, assembled, executed, and debugged step by step. It also allows register and memory inspection [5]. The practical notes also state that students should assemble code, run it, and inspect registers, memory, and flags to verify behaviour [3].

For Part B, the operating system notes explain that the OS manages limited resources such as CPU and RAM. When many processes are ready, the CPU scheduler decides which one runs next.

4. Part A: ARM Assembly Code

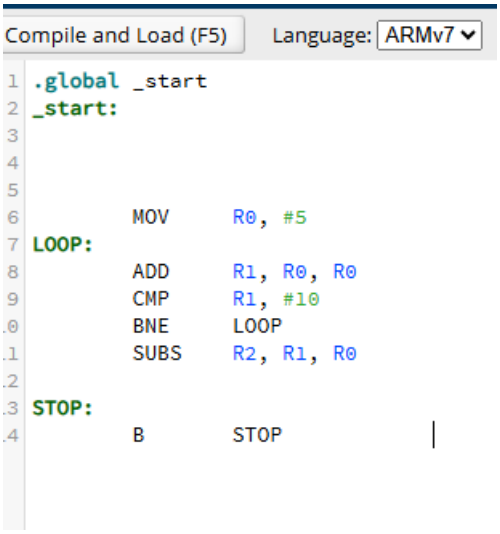

Figure 1 ARM code entered in CPUlator.

5. Explanation of ARM Code

The instruction MOV R0,#5 places the immediate value 5 into register R0. The lecture notes explain that immediate values are constants written directly in the instruction, such as MOV R0,#10 [5] . The instruction ADD R1, R0,R0 adds R0 to itself and stores the result in R1. Since R0 is 5, R1 becomes 10. The notes show that ARM instructions follow the format: Opcode Destination, Operand1, Operand2

The instruction CMP R1,#10 compares R1 with 10 and updates the CPSR flags. The notes explain that ARM uses flags such as N, Z, C, and V for conditional execution [5]. Since R1 equals 10, the Z flag becomes 1. Therefore, BNE LOOP is not taken. The instruction SUBS R2,R1,R0 then stores 5 in R2 and updates the flags.

6. ARM 32-bit Encoding

ARM instructions are usually 32 bits long. The lecture notes show that the instruction fields include condition code, opcode, destination register, operand register, and Operand2 [5].

Table 1

Bits	Field	Meaning
31–28	Condition	Decides when instruction executes
27–20	Opcode/control	Operation type
19–16	Register field	Destination or related register
15–12	Register field	Operand register
11–0	Operand2	Immediate or register operand

Table 2 Encoding Table

Instruction	Condition	Main Operation	Registers / Operand	Hex Encoding
MOV R0,#5	AL	Move immediate	Rd = R0, Operand = 5	E3A00005
ADD R1,R0,R0	AL	Add	Rn = R0, Rd = R1, Rm = R0	E0801000
CMP R1,#10	AL	Compare	Rn = R1, Operand = 10	E351000A
BNE LOOP	AL	Branch not equal	Branch offset	Address dependent
SUBS R2,R1,R0	AL	Subtract with flags	Rn = R1, Rd = R2, Rm = R0	E0512000

The default condition code is AL, meaning the instruction always executes. The instruction BNE uses NE, meaning it only branches when the compared values are not equal.

7. CPUlator Verification

Table 3

Step	Instruction	R0	R1	R2	Result
1	MOV R0,#5	5	_	_	R0 = 5
2	ADD R1,R0,R0	5	10	_	R1 = 10
3	CMP R1,#10	5	10	_	Z flag =1
4	BNE LOOP	5	10	_	Branch not taken
5	SUBS R2,R1,R0	5	10	5	R2 = 5

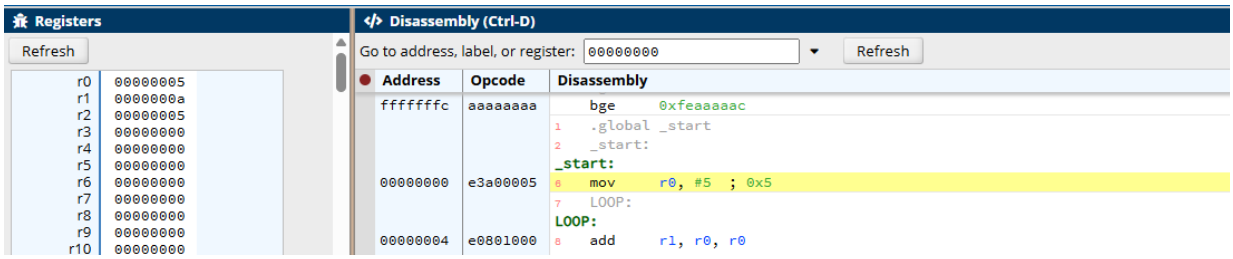

Figure 2 CPUlator register values after MOV and ADD

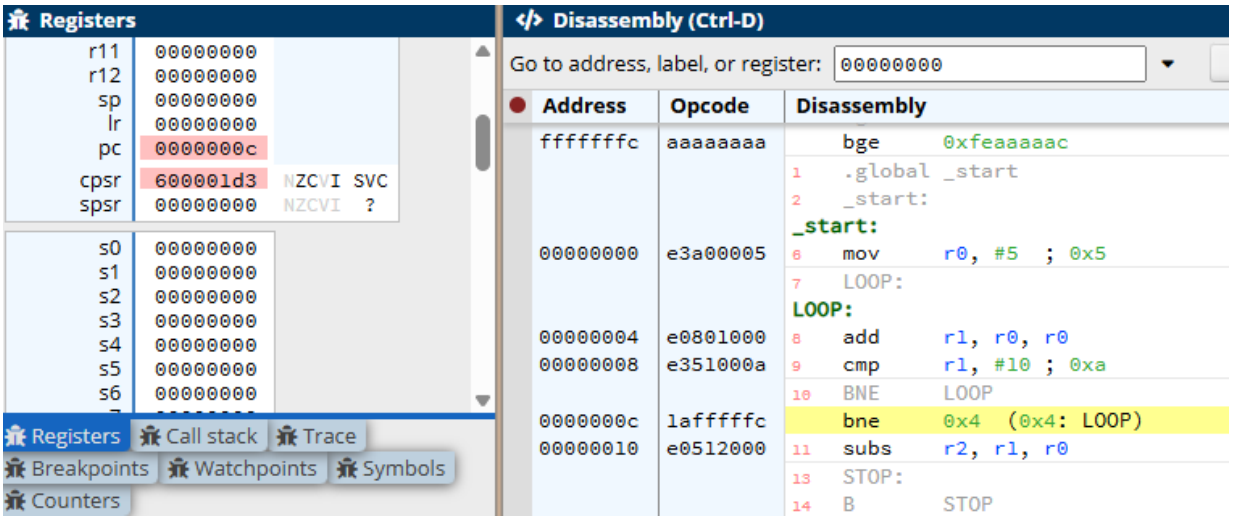

Figure 3 CPUlator CPSR flags after CMP.

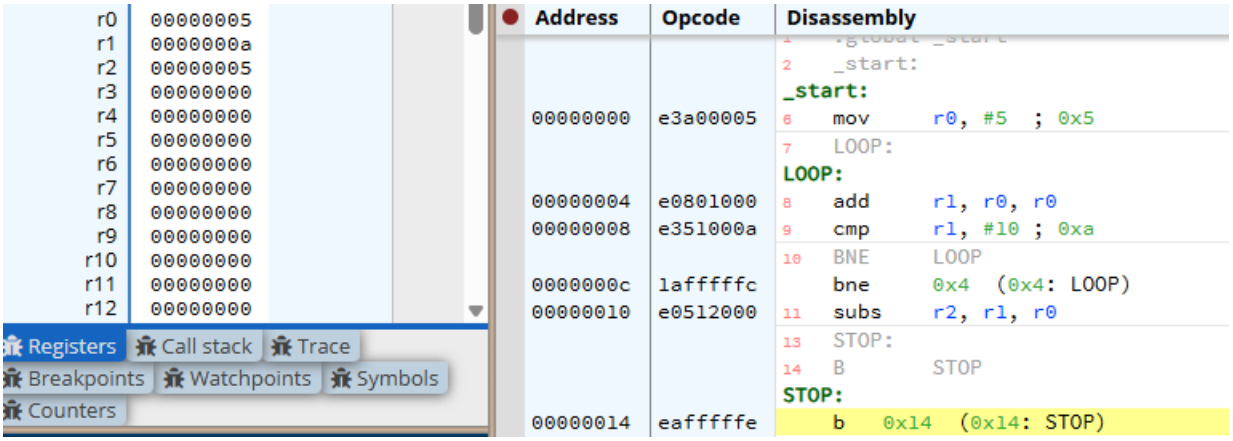

Figure 4 Final CPUlator result showing R2 = 5.

The CPUlator result is correct because the register values match the expected calculation. R0 is 5, R1 is 10, and R2 is 5. The Z flag becomes 1 after the comparison because R1 equals 10.

8. Part B: Python Scheduling Code

```python
from collections import deque

programs = [
    ("P1", 6),
    ("P2", 4),
    ("P3", 8),
    ("P4", 5)
]

def fcfs(programs):
    time = 0
    results = []
    gantt = []

    for name, burst in programs:
        start = time
        time += burst
        finish = time
        waiting = start
        turnaround = finish

        gantt.append((name, start, finish))
        results.append((name, waiting, turnaround))

    return gantt, results
```

```python
def round_robin(programs, quantum):
    queue = deque([[name, burst] for name, burst in programs])
    finish_time = {}
    time = 0
    gantt = []

    while queue:
        name, remaining = queue.popleft()
        run_time = min(quantum, remaining)

        start = time
        time += run_time
        finish = time
        remaining -= run_time

        gantt.append((name, start, finish))

        if remaining > 0:
            queue.append([name, remaining])
        else:
            finish_time[name] = time

    results = []
    for name, burst in programs:
        turnaround = finish_time[name]
        waiting = turnaround - burst
        results.append((name, waiting, turnaround))
```

and palette (Ctrl+Shift+P)

```python
    return gantt, results

def summary(results):
    avg_wait = sum(r[1] for r in results) / len(results)
    avg_turn = sum(r[2] for r in results) / len(results)
    return avg_wait, avg_turn

fcfs_gantt, fcfs_results = fcfs(programs)
rr2_gantt, rr2_results = round_robin(programs, 2)
rr4_gantt, rr4_results = round_robin(programs, 4)

print("FCFS:", summary(fcfs_results))
print("RR q=2:", summary(rr2_results))
print("RR q=4:", summary(rr4_results))
```

Figure 5 Python scheduling code.

```
FCFS: (8.5, 14.25)
RR q=2: (12.75, 18.5)
RR q=4: (12.0, 17.75)
```

Figure 6 Python output for FCFS and Round Robin.

9. Scheduling Results

Algorithm	Average Waiting Time	Average Turnaround Time
FCFS	8.5	14.25
Round Robin q = 2	12.75	18.5
Round Robin q = 4	12.0	17.75

For this workload, FCFS gives the lowest average waiting time. However, Round Robin is fairer because every program gets CPU time in turns. The OS notes explain that Round Robin gives every process a fixed time slice called a quantum, and if the process is not finished, it goes to the back of the queue [4]

10. Integration of Part A and Part B

Part A shows the hardware-level view. ARM instructions are encoded into 32-bit machine code and then executed by the CPU. The instruction decoder reads the opcode bits and decides the operation, operands, and destination register. Part B shows the operating system view [5]. The scheduler decides which program or instruction stream gets CPU time. The OS notes explain that context switching saves CPU state such as R0–R15, CPSR, PC, and SP before another process runs [4].

Therefore, Part A explains what the CPU executes, while Part B explains when each program gets CPU time.

11. GA1 Alignment

GA1 Requirement	**How It Was Met**
Problem identification	The project identifies instruction encoding and CPU scheduling as the main problem.
First principles	ARM registers, opcodes, flags, and scheduling theory were used.
Analysis and modelling	ARM encoding and Python scheduling were modelled and tested.
Engineering knowledge	CPUlator, ARM instruction format, and OS scheduling concepts were applied.
Evaluation	Register results and scheduling results were checked against expected behaviour.
Conclusion	The conclusion is based on CPUlator and Python output

12. Limitations

This project only uses five ARM instructions, so it does not cover all instruction formats. The BNE branch encoding also depends on the exact label address in CPUlator. The Python scheduler is simplified because all programs arrive at time zero, and real context-switching overhead is not included.

13. Conclusion

This project showed how ARM instructions are encoded and how an operating system schedules instruction streams. In Part A, the ARM code was tested in CPUlator and gave the correct register results. In Part B, FCFS and Round Robin were simulated in Python. FCFS had the lowest average waiting time for this example, but Round Robin was fairer because each program received CPU time in turns. The project satisfies GA1 because it defines the problem, applies ARM and OS principles, analyses the results, and gives a conclusion supported by testing.

References

[1] University of Zululand, “GA Assessment 5EEE481,” Department of Engineering, 2026.

[2] University of Zululand, “Projects 5EEE481 GA1: Individual Project Complete Guide,” Department of Engineering, 2026.

[3] University of Zululand, “Lecture 4: ARM Assembly Exercises Using CPUlator,” Department of Engineering, 2026.

[4] University of Zululand, “Lectures 5 and 6: Operating Systems Principles,” Department of Engineering, 2026.

[5] University of Zululand, “Lectures 2 and 3: ARM Assembling Lecture,” Department of Engineering, 2026.